ULModule4Assignment

October 11, 2026

1 Assignment 4: Applying Methods and Techniques for Data Imputation and Semi-Supervised Learning

For the Module 4 programming assignment, you will discover how supervised and unsupervised learning can be used in conjunction with each other for important data science tasks. You will begin with data imputation and move on to an example of conducting semi-supervised learning with label propagation.

- First, you are going to test out and compare several different imputation strategies to deal with missing values of the *features* in a dataset.
- Second, you will do label propagation on a partially-labelled dataset of mushrooms to determine their edibility and compute the accuracy of the label propagation prediction (i.e. the accuracy of mushrooms being labeled as edible/unedible).

[20]:
```
# First import some necessary libraries
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
```

All import statements are provided for you- please do not import other libraries in your own code.

2 Task 1: Data imputation

Machine learning is used throughout the food industry, including in large-scale fruit and vegetable processing to automatically detect and filter acceptable vs unacceptable items for consumers. In this task, we're going to take a look at a simple version of a dataset that might be used to train such classification systems.

We'll work with a database about apples: each row of the table corresponds to an observed apple, and the columns corresponding to different features of an apple such as its size, color, shape, etc. There is also a column that contains a binary label marking each observed apple as "good" or "bad" for consumption.

Real-world datasets often have missing data, so the goal for this task is for you to get some practice using the imputation methods in Module 4 to infer missing values in a dataset. Please refer to the module 4 reading, notebook and data imputation lecture for examples of code on how to use an Imputer class (including optionally tied to a Regression class via a pipeline object).

Our original apples dataset doesn't have any missing data, so we produce a noisy version of the original dataset for you to work with that has random features missing for random apples. Missing data in a cell is represented by `NaN`. The provided function `create_copy_with_missing_values` is the code that produces the noisy version of the apple dataset.

Your goal for Task 1 is to compare the effectiveness of different imputer strategies for filling in the missing data cells (`NaN`) in the noisy apple dataset. We're going to do a *task-based* evaluation of imputer effectiveness, where the task is correctly predicting an apple's "goodness" score: a superior imputation strategy should give a higher prediction score on test data.

For all the subtasks below, the array `X_missing` contains the noisy dataset features of the apples, such as size, shape, etc. and `y_missing` contains the binary good/bad label for each apple in the noisy dataset.

[22]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import SimpleImputer, IterativeImputer, KNNImputer
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.datasets import load_diabetes
def create_copy_with_missing_values(X, y, missing_rate=0.75):
    n_samples, n_features = X.shape

    n_missing_samples = int(n_samples * missing_rate)

    # now pick which rows, columns should correspond to the missing samples
    missing_instances = rng.choice(n_samples, size=n_missing_samples,␣
 ↪replace=False)
    missing_features = rng.randint(0, n_features, n_missing_samples)

    X_missing = X.copy()
    y_missing = y.copy()

    X_missing[missing_instances, missing_features] = np.nan

    return X_missing, y_missing

rng = np.random.RandomState(42)
CV_SPLITS = 7
missing_rate = 0.70
regressor = RandomForestRegressor(random_state=42)
x_labels = [
    "Simple imputation: always zero",
    "Removing Rows with empty values",
    "k-NN imputation",
        "Full data",
]
```

```
score_type = "r2"
n_entries = len(x_labels)
mses_dataset = np.zeros(n_entries)
stds_dataset = np.zeros(n_entries)

# Load the apple dataset from the CSV file
apple_data = pd.read_csv('./assets/apple_quality.csv')
apple_data = apple_data.head(100)

# Extract features (X) and target variable (y) from the dataset
X_full = apple_data.drop(columns=['Quality', 'A_id']).values
y_full = apple_data['Quality'].values
X_full = X_full.astype(np.float64)

label_mapping = {'good': 1, 'bad': 0}

# Convert labels in y_full using the mapping function
y_full = np.array([label_mapping[label] for label in y_full])

print(X_full)
print(y_full)

X_missing, y_missing = create_copy_with_missing_values(
    X_full, y_full, missing_rate=missing_rate
)

def get_scores_using_imputer(imputer):
    estimator = make_pipeline(imputer, regressor)
    impute_scores = cross_val_score(
        estimator, X_missing, y_missing, scoring=score_type, cv=CV_SPLITS
    )
    # print(impute_scores)
    return impute_scores
```

```
[[-3.97004852 -2.51233638  5.34632961 -1.01200871  1.84490036  0.3298398
  -0.49159048]
 [-1.19521719 -2.83925653  3.66405876  1.58823231  0.8532858   0.86753008
  -0.72280937]
 [-0.29202386 -1.35128199 -1.73842916 -0.34261593  2.83863551 -0.03803333
   2.62163647]
 [-0.65719577 -2.27162661  1.32487385 -0.09787472  3.63797049 -3.41376134
   0.79072322]
 [ 1.36421682 -1.29661188 -0.38465821 -0.55300577  3.03087435 -1.30384943
   0.50198404]
 [-3.42539975 -1.4090822  -1.9135112  -0.55577486 -3.85307147  1.91461592
  -2.98152317]
 [ 1.33160574  1.63595571  0.87597424 -1.67779794  3.10634445 -1.84741673
```

```
  2.41417051]
[-1.9954621  -0.42895848  1.53064358 -0.74297168  0.158834    0.97443786
 -1.47012507]
[-3.86763223 -3.73451358  0.98642907 -1.20765455  2.29287292  4.08092079
 -4.87190476]
[-0.72798271 -0.44282035 -4.09222283  0.59751292  0.39371426  1.62085677
  2.18560772]
[-2.69933629 -1.32950699 -1.41850685 -0.62554577  2.37107437  3.40316452
 -2.81080817]
[ 2.45095984 -0.56417741 -1.63504073  0.94239987 -2.08731667  1.21432169
  1.29432393]
[-0.17081172 -1.86727114 -1.77184473  2.41315532 -3.09455474 -0.62488438
 -2.076114  ]
[-1.34553054 -1.62370112  2.04414375  1.75481293  0.99756709  0.43417986
  1.72402608]
[ 2.83958094 -0.34479819 -1.01979729  0.89458086 -1.30006088  0.58237862
  1.70970821]
[-2.65988739 -2.79568421  4.23040359  0.6975504   2.1809111  -0.0887754
 -1.08362079]
[-1.46895155 -1.95035959 -2.21437289  0.90975851  2.86444885  3.96595566
 -0.55820868]
[-0.07437018 -4.71474995  0.24976764  2.93531906  1.4097551  -2.64381021
  1.25097035]
[-0.30236427  1.72439585 -2.44233722  3.46510848  0.44979168 -0.07436245
  2.49378199]
[-2.1080499   0.3564674  -1.15619328  4.32672252  1.56154312 -4.63017426
 -1.37665721]
[-2.3345895  -2.9437089  -3.45262772  0.76239173  4.07646237  6.34644535
  0.72677567]
[ 1.17759278 -0.72165449 -1.38711622  7.6198518   1.06928838 -3.73480541
  2.64294824]
[-2.42394622 -0.69850065  0.14603041  0.63010617  2.99056051  0.7794729
  3.18418819]
[ 0.1357142  -0.75375686 -2.19614646  1.03927609  0.58053755  0.22730878
  2.08661854]
[ 0.52296076 -1.42808524 -0.74351874  1.78671619 -4.20754355 -1.82523122
 -1.43042986]
[-1.29946839 -3.50479201 -1.129402    0.55590538 -2.80755016  1.71463011
 -3.84664182]
[-0.30069828 -0.51360325  0.92100624  1.37817207  2.27474669  0.74533638
 -2.93402889]
[ 1.99983096  0.6699898  -2.09961599  2.64581897 -0.98949589  0.37333013
 -2.3976918 ]
[ 1.44605205  1.65669182 -1.77752075  1.70896221 -0.34167032 -1.32243949
 -1.15812807]
[-0.59582548  1.53467991 -1.76931897  0.66450184 -0.43287445  0.80043869
  1.64595436]
[ 0.41049215 -2.26029006 -3.33594802 -0.28520519 -2.73384426  2.58598571
```

```
 -3.03691971]
[-2.56087155 -0.0964117   2.4137741   1.31829367  3.27168813 -1.79718456
 -2.63684481]
[ 1.68166262 -2.9816738  -0.07980509 -0.94786639  3.03481256 -0.70861123
  1.16697074]
[-0.58879603 -1.12198715  2.32429509  0.31193071  5.14873942 -3.35198841
  5.56010869]
[-0.03436794 -1.3328051   2.32910127  2.8627589   0.82523931  1.06267209
  2.30033053]
[-0.95540234 -2.46109678 -4.08715617  0.83910762  1.00321619  4.97563997
 -0.98138767]
[-3.06116527  0.83710364 -1.07663823  1.52473801  1.76414026 -0.05647271
 -1.49511262]
[ 3.94811371 -1.63418308 -4.77378012  1.9316313   0.79053969  1.5830972
 -2.47805593]
[-1.79671851 -3.46922729  3.68543676 -0.16414062  1.32029846  1.48010953
 -2.29296443]
[ 1.28673791 -2.42952453  0.9561331   1.47754224  4.78637568 -3.3823574
  2.51934692]
[ 1.19654503 -1.44810429 -0.92151676  2.00549129  0.40474148 -0.70077885
  1.07112971]
[-0.35419034 -2.02404253  1.36698489  3.23361617  0.99266852  1.52560855
  2.74092476]
[-1.53860666 -2.74784439  1.84495006  0.49124907 -0.42581552  1.25976818
 -1.68092409]
[ 0.8078541   0.75270701  0.17922635  1.45158218 -1.61185917 -2.77627496
  0.18490561]
[ 3.93036139  2.75440649 -2.66794954  1.42824817  0.09276123  1.09579366
  2.62727416]
[ 0.03347095 -0.04416096 -0.48120247  4.93720951 -0.6098729  -3.23150403
  1.24879179]
[-0.48736669 -0.76880532 -0.77923151  2.02525662  1.47792698  0.26778807
 -4.45457017]
[-1.38799018  2.73851076 -2.34921685  0.95343849  1.86071572  2.31730991
  3.09781838]
[-3.3285715   0.65257694 -1.42440296  0.72738538  1.57692135  2.57780352
  0.65067787]
[-1.10706015  2.26119823 -2.63519702  0.88226478 -0.02399536  2.21809937
  2.86504837]
[-2.93861682  0.52445492  0.80225744 -1.47593088 -0.33634894  1.43064807
 -2.29887552]
[-0.41130768 -1.75859528 -2.21386478 -0.14953526  1.5761843   0.92919741
  1.41698309]
[ 1.34376332 -2.51441714 -3.46153894  3.84115419  0.28051514 -0.02543681
  0.85622891]
[ 1.2054503  -1.78350847  0.65268306  2.44298343  1.34691458  1.11506216
 -1.19865698]
[ 3.01034428 -3.42302863 -3.11097796 -1.54304451  3.56760819  1.6962326
```

```
  0.4458404 ]
[ 2.03279708 -0.64102136 -2.40968992  2.93612829  1.67548455 -0.90458854
  2.28735035]
[ 1.82265732 -0.5456173  -0.36782084  0.25298329 -0.78910741  2.13591178
 -3.3439558 ]
[ 1.09848682 -1.02037418  1.21113118  1.26017767 -1.02990271  2.62273799
  2.66254783]
[-0.47988303 -0.0791954  -0.84354704  3.04956761  3.66383812 -0.36569866
 -1.47353753]
[ 0.83309773  0.44971363 -1.63854887 -0.63133803  2.83090637 -0.66886038
  1.0494288 ]
[-3.46415534  1.35211115  0.45802572  0.89274674  3.12981846 -1.74467453
 -2.20461073]
[-4.67414092  0.79704588 -0.89721267 -1.39398317  0.4921298   3.20170779
 -0.58628671]
[-3.34149305 -1.35555654  1.43364668  1.72235668  0.11291181  0.68878842
  1.6191837 ]
[ 0.45531261 -2.38289545 -0.83742965 -2.36617514 -0.27646012  0.85233525
 -0.03112605]
[-2.23618812 -1.17456106  1.48270348 -0.27716106  1.49759843  1.07573139
 -0.93697059]
[-3.56789797  1.04374218  2.56339958  0.43727671  1.67851822 -5.31383822
  2.07198361]
[-2.18893279  2.13175426  0.26084129  3.88604837 -3.19626311 -3.56059388
  0.08951294]
[-1.56976389  1.05986497  1.5188801   1.66609628  3.06893135 -1.48035112
  2.76945861]
[-1.1290609  -0.25783617 -1.94223891 -0.02093466 -0.72413892  2.66211102
 -0.11247289]
[-0.39960862 -3.19817457 -1.25063926  2.90927768 -2.3140371  -0.23111205
 -3.02939237]
[ 0.61013244 -3.46081867  1.33675931  0.3874515  -0.13678123  2.98864555
 -2.04476895]
[ 2.35440229 -2.66027025  2.6154159  -3.29121025  4.60445412 -1.24011742
  3.68958086]
[ 0.56285123 -1.82798915 -4.62224368 -0.32573547  2.57880492  2.48154138
  1.15276855]
[ 0.03941369 -1.13743024  0.47773233  1.65648057 -3.25623203 -0.50358422
  0.5286988 ]
[-2.15961039 -1.29595949 -1.73243521  0.08072449 -1.95961421  2.72303972
 -1.47788311]
[ 2.16059155 -1.60812751 -3.14953998  1.35425844  2.16930878  0.56145539
  1.46604361]
[ 2.24978205  1.30681125 -3.3621429  -3.29234302  1.53637332  2.32269389
  0.17170376]
[ 0.91336708  0.23822763 -0.86645313  0.41107897  0.77724708 -0.69055768
  0.26908152]
[-2.17806896  0.71106002 -2.72352818 -0.79461916  1.37163224  2.66166924
```

```
 -1.43801903]
[-2.1674624  -1.72966774  1.29234408  2.34302702  0.313248   -0.90759004
 -1.78744711]
[ 1.90689525 -0.28655704 -0.64227627 -0.9096866   0.56727051  0.44767672
  2.331285  ]
[-2.45137374 -2.46044536  3.31055959  1.10007227  1.66725622 -0.01057985
 -0.90836025]
[ 2.35504706 -0.80500708 -4.04686924 -0.62127355  2.72101688  1.40361245
 -1.73511771]
[ 1.41034947 -6.23510704  0.60667877  2.78152646  0.18841928 -1.65556849
  1.78746561]
[ 1.75626858 -2.07824612 -1.34700307  1.57070571  0.20559484  3.33230319
 -0.1598795 ]
[-0.49031354 -2.69359424 -2.26292813  1.4082187   0.89413031  4.09705263
 -1.19297578]
[ 1.88965188 -1.23488459 -2.00322687  1.61822028  0.36072433  2.71221359
  0.12491436]
[ 0.52355109 -0.78529343  0.30717746 -0.77388239  1.22044626 -1.32970236
  1.35548521]
[-1.44761768 -0.40082252 -2.01581936  0.51914766  0.92156357  2.63835347
 -0.52902919]
[ 1.28831547  0.11832405 -2.03989032  1.25420755  0.42178058  0.27382252
  0.06435217]
[ 0.51694812  0.64378051 -2.75287795  1.26593823  0.71429018  1.04106413
 -2.96741598]
[-0.62954061 -1.16111198 -0.33009969  0.93046988  2.08748948  0.36954986
 -4.56090534]
[-0.69171312 -2.43134029 -0.90966355  2.42832065  0.24207936  2.31357557
  1.28594294]
[-2.09043052 -2.26131964  3.82271392 -0.51214085  0.66059513  0.66342337
 -1.71408531]
[ 0.09739757 -2.09013966 -5.01888909  0.17339726  1.52466235  5.27844833
 -1.70039607]
[ 1.37260262 -0.81619118 -1.13803325  1.94895657 -0.22223192 -0.56191532
  1.65215817]
[-0.61811189 -2.13170354 -1.58002519 -0.19531631  3.62567676  3.2593094
 -1.63364199]
[ 1.80336603 -0.97604405 -1.54691943  0.9263053   3.13968117 -0.68322271
  3.40145538]
[-0.20134277  0.57913176 -2.12603534  3.12928488 -4.40184891 -0.55262758
 -2.24996583]
[-1.15932428  0.72317207 -0.04025359 -0.74829645  0.67795748  1.25184885
 -2.14602339]]
[1 1 0 1 1 0 1 1 0 0 0 1 0 1 1 1 0 1 0 1 0 1 0 0 0 0 1 0 0 0 0 1 1 0 1 0 1
 1 1 1 0 1 0 0 1 1 1 0 0 0 1 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 0 0 0 0 1 0 0
 0 1 1 0 0 0 1 1 0 1 1 0 1 1 0 0 1 1 0 1 0 1 0 1 1 1]
```

3 Task 1a: Simple imputation by replacing with zero

One strategy to deal with missing values is when we come across with an apple that might have a missing attribute such as "weight" or "size", to just simply replace that missing value with zero. In general, as we covered in the lecture, this is a risky strategy that can introduce significant bias into the data if the true values are often non-zero. However, we'll use this very basic strategy as a baseline against which to compare the other imputation strategies.

In this subtask, you will be writing a function that first creates a SimpleImputer with the following arguments:

`missing_values = np.nan, add_indicator=True, strategy = "constant", fill_value = 0.`

Then pass this simple imputer object as input to the provided `get_scores_using_imputer` function defined above to obtain the mean and standard deviation of the imputation scores, and return both of those numbers from your function, in that order. The mean and standard deviation can be obtained by calling `.mean()` and `.std()` respectively on the object returned by the `get_scores_using_imputer` function.

[23]:
```
task_id = '1a'
```

[24]:
```
def task_1a_solution():

    mean = 0
    std = 0
    imputer = SimpleImputer(missing_values=np.nan, add_indicator=True, strategy␣
 ↪= "constant", fill_value = 0)
    # YOUR CODE HERE
    #raise NotImplementedError()
    impute_scores = get_scores_using_imputer(imputer)
    #print(impute_scores.shape)
    mean, std = impute_scores.mean(), impute_scores.std()

    return mean, std
```

[25]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_1a_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "
```

```
del stu_ans
```

```
Task 1a - AG tests
Task 1a - your answer:
(0.14868134920634932, 0.19039241486251157)
```

4 Task 1b: Removing rows with missing values

Another strategy to deal with missing values is to simply remove rows with missing values.

In the function, first use `np.isnan` on `X_missing` to find the indices of rows with any missing values, and then use that result to make new versions of `X_missing` and `y_missing` that don't contain those rows. We'll call these new versions `X_cleaned` and `y_cleaned`.

Next, call `cross_val_score` as in the `get_scores_using_imputer` utility code above, but this time, you will be passing in the provided estimator `new_pipeline`, and `X_cleaned` and `y_cleaned` instead of `X_missing` and `y_missing`. All other arguments will remain the same. As in Task 1a, you then call `.mean()` and `.std()` on the object returned by `cross_val_score` to obtain the mean and standard deviation of the imputation scores, and return both of those numbers from your function, in that order.

[26]:
```
task_id = '1b'
```

[27]:
```
def task_1b_solution():

    mean = 0
    std = 0
    new_pipeline = make_pipeline(regressor)
    # YOUR CODE HERE
    #raise NotImplementedError()
    #indices = np.where(np.isnan(X_missing).any(axis=1))[0]
    #print(indices)
    mask = np.isnan(X_missing).any(axis=1)
    #print(mask)
    X_cleaned, y_cleaned = X_missing[~mask], y_missing[~mask]
    impute_scores = cross_val_score(
        new_pipeline, X_cleaned, y_cleaned, scoring=score_type, cv=CV_SPLITS
    )
    mean, std = impute_scores.mean(), impute_scores.std()
    return mean, std

# Call the function
#task_1b_solution()
```

[28]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_1b_solution()
```

```
print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "




del stu_ans
```

```
Task 1b - AG tests
Task 1b - your answer:
(0.15484047619047622, 0.3114090835063252)
```

5 Task 1c: Using k-NN imputation

A third strategy for imputing missing values is to use a more refined imputation method: k-NN imputation. This approach uses the k-nearest neighbors algorithm to find similar nearby data points and then estimate the missing value using the mean or median of the values of that feature from the nearest neighbor points.

Start by creating a KNNImputer object with `missing_values=np.nan` and `add_indicator=True`. (We covered the purpose of the `add_indicator` option in the imputation lecture.)

Then, as in Task 1a, use the KNNImputer objects as the input argument to the provided `get_scores_using_imputer` function to obtain the mean and standard deviation scores that are output from the imputer, and return both of those numbers from your function, in that order.

[29]:
```
task_id = '1c'
```

[30]:
```
def task_1c_solution():

    mean = 0
    std = 0

    # YOUR CODE HERE
    #raise NotImplementedError()

    imputer = KNNImputer(missing_values=np.nan, add_indicator=True)
    impute_scores = get_scores_using_imputer(imputer)
    mean, std = impute_scores.mean(), impute_scores.std()

    return mean, std

# Call the function
```

[31]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_1c_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple "




del stu_ans
```

```
Task 1c - AG tests
Task 1c - your answer:
(0.2222202777777778, 0.12328810847604282)
```

With all three imputation methods implemented, run the code below to compare the prediction effectiveness of the different strategies. Which imputation strategy resulted in the best mean prediction performance?

[32]:
```
def get_full_score():
    full_scores = cross_val_score(regressor, X_full, y_full,␣
 ↪scoring=score_type, cv=CV_SPLITS)
    return full_scores.mean(), full_scores.std()


### Plot results
### This plotting code adapted from scikit-learn impute documentation.
mses_dataset[0], stds_dataset[0] = task_1a_solution()
mses_dataset[1], stds_dataset[1] = task_1b_solution()
mses_dataset[2], stds_dataset[2] = task_1c_solution()
mses_dataset[3], stds_dataset[3] = get_full_score()

n_bars = n_entries
xval = np.arange(n_bars)
colors = ["r", "g", "b", "orange"]

plt.figure(figsize=(12, 6))
ax1 = plt.subplot(121)

ax1.set_title("Effect of different imputation methods on regression score")
ax1.set_xlim(left=np.min(mses_dataset) * 0, right=np.max(mses_dataset) * 2)
ax1.set_yticks(xval)
ax1.set_xlabel(score_type)
ax1.invert_yaxis()
```

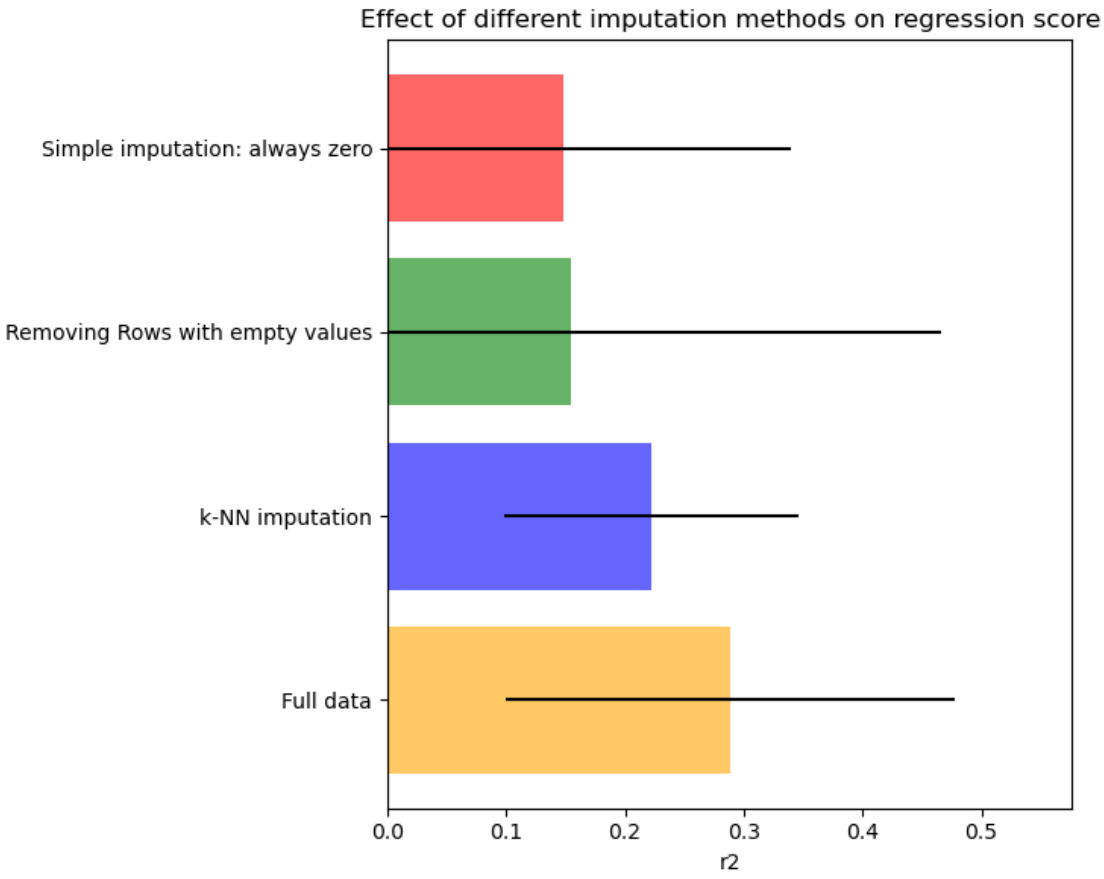
```
ax1.set_yticklabels(x_labels)

for j in xval:
    ax1.barh(
        j,
        mses_dataset[j],
        xerr=stds_dataset[j],
        color=colors[j],
        alpha=0.6,
        align="center",
    )

print(mses_dataset, stds_dataset)

plt.tight_layout()
```

```
[0.14868135 0.15484048 0.22222028 0.28818417] [0.19039241 0.31140908 0.12328811
0.18864577]
```



6 Task 2: Label Propagation

(40 points)

Unluckily, you have become stranded on a desert island. But luckily, there is food on the island in the form of mushrooms, which grow everywhere. Also luckily, whatever hapless individual was stranded here before you kept a record of every type of mushroom growing on this island with features such as cap shape, odor, and most importantly, whether or not they are edible or poisonous. Unluckily, however, some of the stickers labelling the mushrooms as edible or poisonous have eroded over time! Fortunately, you managed to save your solar-powered laptop and have it with you, so to make the mushrooms (and yourself) last as long as possible before help arrives, you decide to do label propagation on this damaged mushrooms dataset to determine the edibility of unlabeled mushrooms. You only want to eat mushrooms labeled by label propagation as edible if you're confident that your algorithm is accurate. Your task is to calculate the effectiveness of the label propagation algorithm on your mushroom data by computing the downstream accuracy of a classifier trained with the inferred labels, i.e. the fraction of mushrooms being correctly labeled as edible/unedible. The classifier in this case used to predict a mushroom's edibility will be a linear support vector machine (SVC).

To use label propagation, as explained in the lecture, we first need to define the distance measure to be used on pairs of items. Our items are mushrooms, which are represented using a mixture of different feature types (e.g. categorical, numeric). In this case, a very handy distance measure for mixed-type features is the Gower distance measure. We provide code to compute Gower's distance below.

To accomplish this, do the following:

- Run the preamble code below. This will, among other things, create a train-test split of the data and set variables `X_train`, `X_test`, `y_train`, `y_test`.
- As provided in the solution preamble, you call `get_mangled_label_dataset` with the specified parameters, on `X_train` to produce `X_train_complete`, and `y_train_complete_missing`, which represent a simulated dataset with missing labels.
- Perform label propagation by first creating a [LabelSpreading] object (https://scikit-learn.org/stable/modules/generated/sklearn.semi_supervised.LabelSpreading.html) object with `kernel='knn', n_neighbors=9, max_iter=30, tol=0.001`
- Then fit the label propagation model using `X_train_complete`, and `y_train_complete_missing`.
- Use `.predict()` to obtain the predicted labels that fill in the missing `y` label values.
- Then, create a support vector classifier **using the [SVC] class (https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html)** with default parameters and fit the classifier using the complete training set with inferred labels from the previous step.
- Compute the accuracy of the SVC classifier on the test set (`y_test` has the gold standard labels, to be compared against the predicted labels from the SVC classifier for the items in `X_test`)
- Your function should return a single float: the accuracy of the SVC classifier.

Note: the two cells below contain very long preamble code. You may want to minimize them to make it easier to work between the instructions and your solution code.

[33]:
```
"""


"""
from sklearn import datasets
from sklearn.semi_supervised import LabelPropagation
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score


def get_mangled_label_dataset(X_train, y_train, frac_0, frac_1):
    """
    This function takes a dataset and label set as input, and returns the␣
 ↪following:
      - The X features of the subset of instances that had intact labels
      - The y labels of the subset of instances that had intact labels
      - The X features of the complete training set
      - The y labels of the complete training set WITH DESTROYED LABELS␣
 ↪according to the assignment specifications.

    input parameters frac_0, frac_1 indicate the probability you want to mangle␣
 ↪class 0 and class 1 labels
    respectively.
    """

    rng = np.random.RandomState(0)

    X_train_0 = X_train[y_train == 0]
    y_train_0 = y_train[y_train == 0]

    X_train_1 = X_train[y_train == 1]
    y_train_1 = y_train[y_train == 1]

    X_train_complete = np.vstack((X_train_0, X_train_1))
    y_train_complete = np.concatenate((y_train_0, y_train_1))

    random_unlabeled_points_0 = rng.rand(len(y_train_0)) < frac_0
    random_unlabeled_points_1 = rng.rand(len(y_train_1)) < frac_1
    random_unlabeled_points = np.concatenate(
        (random_unlabeled_points_0, random_unlabeled_points_1)
    )

    random_labeled_points_0 = np.logical_not(random_unlabeled_points_0)
    random_labeled_points_1 = np.logical_not(random_unlabeled_points_1)
```

```
    labeled_subset_X_train_0 = X_train_0[random_labeled_points_0]
    labeled_subset_X_train_1 = X_train_1[random_labeled_points_1]
    labeled_subset_y_train_0 = y_train_0[random_labeled_points_0]
    labeled_subset_y_train_1 = y_train_1[random_labeled_points_1]

    labeled_subset_X_train = np.vstack(
        (labeled_subset_X_train_0, labeled_subset_X_train_1)
    )
    labeled_subset_y_train = np.concatenate(
        (labeled_subset_y_train_0, labeled_subset_y_train_1)
    )

    labels_train_with_destroyed = np.copy(y_train_complete)
    labels_train_with_destroyed[random_unlabeled_points] = -1

    return (
        labeled_subset_X_train,
        labeled_subset_y_train,
        X_train_complete,
        labels_train_with_destroyed,
    )
```

[34]:
```
from scipy.spatial import distance
from sklearn.utils import validation
from sklearn.metrics import pairwise
from scipy.sparse import issparse

# The following functions are provided by Marcelo Beckmann and currently are␣
 ↪part of the scikit-learn
# github repository but have not yet been officially released.


# This is a utility function used by gower_distances below.
def check_pairwise_arrays(X, Y, precomputed=False, dtype=None):
    X, Y, dtype_float = pairwise._return_float_dtype(X, Y)

    warn_on_dtype = dtype is not None
    estimator = "check_pairwise_arrays"
    if dtype is None:
        dtype = dtype_float

    if Y is X or Y is None:
        X = Y = validation.check_array(
            X, accept_sparse="csr", dtype=dtype, estimator=estimator
        )
    else:
        X = validation.check_array(
```

```
            X, accept_sparse="csr", dtype=dtype, estimator=estimator
        )
        Y = validation.check_array(
            Y, accept_sparse="csr", dtype=dtype, estimator=estimator
        )

    if precomputed:
        if X.shape[1] != Y.shape[0]:
            raise ValueError(
                "Precomputed metric requires shape "
                "(n_queries, n_indexed). Got (%d, %d) "
                "for %d indexed." % (X.shape[0], X.shape[1], Y.shape[0])
            )
    elif X.shape[1] != Y.shape[1]:
        raise ValueError(
            "Incompatible dimension for X and Y matrices: "
            "X.shape[1] == %d while Y.shape[1] == %d" % (X.shape[1], Y.shape[1])
        )

    return X, Y


# gower_distances: see header below for details.
#
# For purposes of this assignment you can simply call it like this on a dataset␣
 ↪X (n instances):
# D = gower_distances(X)
#
# which results an n x n distance matrix.
#
# Source: M. Beckmann  https://github.com/scikit-learn/scikit-learn/pull/9555


def gower_distances(X, Y=None, feature_weight=None, categorical_features=None):
    """Computes the gower distances between X and Y

    Gower is a similarity measure for categorical, boolean and numerical mixed
    data.


    Parameters
    ----------
    X : array-like, or pandas.DataFrame, shape (n_samples, n_features)

    Y : array-like, or pandas.DataFrame, shape (n_samples, n_features)

    feature_weight :  array-like, shape (n_features)
```

```
        According the Gower formula, feature_weight is an attribute weight.

    categorical_features: array-like, shape (n_features)
        Indicates with True/False whether a column is a categorical attribute.
        This is useful when categorical atributes are represented as integer
        values. Categorical ordinal attributes are treated as numeric, and must
        be marked as false.

        Alternatively, the categorical_features array can be represented only
        with the numerical indexes of the categorical attribtes.

    Returns
    -------
    similarities : ndarray, shape (n_samples, n_samples)

    Notes
    ------
    The non-numeric features, and numeric feature ranges are determined from X␣
↪and not Y.
    No support for sparse matrices.

    """

    if issparse(X) or issparse(Y):
        raise TypeError("Sparse matrices are not supported for gower distance")

    y_none = Y is None

    # It is necessary to convert to ndarray in advance to define the dtype
    if not isinstance(X, np.ndarray):
        X = np.asarray(X)

    # this is necessary as strangelly the validator is rejecting
    #  numeric arrays with NaN
    # array_type = np.object

    # array_type = object replaces above due to deprecation of
    # np.object.
    array_type = object

    if np.issubdtype(X.dtype, np.number) and (
        np.isfinite(X.sum()) or np.isfinite(X).all()
    ):
        array_type = type(np.zeros(1, X.dtype).flat[0])

    X, Y = check_pairwise_arrays(X, Y, precomputed=False, dtype=array_type)
```

```
    n_rows, n_cols = X.shape

    if categorical_features is None:
        categorical_features = np.zeros(n_cols, dtype=bool)
        for col in range(n_cols):
            # In numerical columns, None is converted to NaN,
            # and the type of NaN is recognized as a number subtype
            if not np.issubdtype(type(X[0, col]), np.number):
                categorical_features[col] = True
    else:
        categorical_features = np.array(categorical_features)

    # if categorical_features.dtype == np.int32:
    # np.int deprecated
    # if np.issubdtype(categorical_features.dtype, np.int):

    if np.issubdtype(categorical_features.dtype, np.int64):
        new_categorical_features = np.zeros(n_cols, dtype=bool)
        new_categorical_features[categorical_features] = True
        categorical_features = new_categorical_features

    # print(categorical_features)

    # Categorical columns
    X_cat = X[:, categorical_features]

    # Numerical columns
    X_num = X[:, np.logical_not(categorical_features)]
    ranges_of_numeric = None
    max_of_numeric = None

    # Calculates the normalized ranges and max values of numeric values
    _, num_cols = X_num.shape
    ranges_of_numeric = np.zeros(num_cols)
    max_of_numeric = np.zeros(num_cols)
    for col in range(num_cols):
        col_array = X_num[:, col].astype(np.float32)
        max = np.nanmax(col_array)
        min = np.nanmin(col_array)

        if np.isnan(max):
            max = 0.0
        if np.isnan(min):
            min = 0.0
        max_of_numeric[col] = max
        ranges_of_numeric[col] = (1 - min / max) if (max != 0) else 0.0
```

```
    # This is to normalize the numeric values between 0 and 1.
    X_num = np.divide(
        X_num, max_of_numeric, out=np.zeros_like(X_num), where=max_of_numeric !
↪= 0
    )

    if feature_weight is None:
        feature_weight = np.ones(n_cols)

    feature_weight_cat = feature_weight[categorical_features]
    feature_weight_num = feature_weight[np.logical_not(categorical_features)]

    y_n_rows, _ = Y.shape

    dm = np.zeros((n_rows, y_n_rows), dtype=np.float32)

    feature_weight_sum = feature_weight.sum()

    Y_cat = None
    Y_num = None

    if not y_none:
        Y_cat = Y[:, categorical_features]
        Y_num = Y[:, np.logical_not(categorical_features)]
        # This is to normalize the numeric values between 0 and 1.
        Y_num = np.divide(
            Y_num, max_of_numeric, out=np.zeros_like(Y_num),␣
↪where=max_of_numeric != 0
        )
    else:
        Y_cat = X_cat
        Y_num = X_num

    for i in range(n_rows):
        j_start = i

        # for non square results
        if n_rows != y_n_rows:
            j_start = 0

        Y_cat[j_start:n_rows, :]
        Y_num[j_start:n_rows, :]
        result = _gower_distance_row(
            X_cat[i, :],
            X_num[i, :],
            Y_cat[j_start:n_rows, :],
            Y_num[j_start:n_rows, :],
```

```
            feature_weight_cat,
            feature_weight_num,
            feature_weight_sum,
            categorical_features,
            ranges_of_numeric,
            max_of_numeric,
        )
        dm[i, j_start:] = result
        dm[i:, j_start] = result

    return dm


def _gower_distance_row(
    xi_cat,
    xi_num,
    xj_cat,
    xj_num,
    feature_weight_cat,
    feature_weight_num,
    feature_weight_sum,
    categorical_features,
    ranges_of_numeric,
    max_of_numeric,
):
    # categorical columns
    sij_cat = np.where(xi_cat == xj_cat, np.zeros_like(xi_cat), np.
 ↪ones_like(xi_cat))
    sum_cat = np.multiply(feature_weight_cat, sij_cat).sum(axis=1)

    # numerical columns
    abs_delta = np.absolute(xi_num - xj_num)
    sij_num = np.divide(
        abs_delta,
        ranges_of_numeric,
        out=np.zeros_like(abs_delta),
        where=ranges_of_numeric != 0,
    )

    sum_num = np.multiply(feature_weight_num, sij_num).sum(axis=1)
    sums = np.add(sum_cat, sum_num)
    sum_sij = np.divide(sums, feature_weight_sum)
    return sum_sij
```

[35]:
```
mushroom_df = pd.read_csv("./assets/mushrooms.csv")

# Read the mushroom dataset
```

```
mushroom_df = pd.read_csv("./assets/mushrooms.csv")

mushroom_df = mushroom_df.head(100)

# Separate features (X) and target variable (y)
X = mushroom_df.iloc[:, 1:]
y = mushroom_df.iloc[:, 0]
y = y.replace({'e': 0, 'p': 1})
X = gower_distances(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

X_train = StandardScaler().fit_transform(X_train)
X_test = StandardScaler().fit_transform(X_test)
```

[36]:
```
task_id = '2'
```

[37]:
```
def task_2_solution():

    accuracy = 0.0
    # Simulate label destruction
    _, _, X_train_complete, y_train_complete_missing =␣
 ↪get_mangled_label_dataset(X_train,
                                                                    ␣
 ↪  y_train,
                                                                    ␣
 ↪    0.7,
                                                                    ␣
 ↪      0.7)


    # YOUR CODE HERE
    #raise NotImplementedError()
    from sklearn.semi_supervised import LabelSpreading
    lp_model = LabelSpreading(kernel='knn', n_neighbors=9, max_iter=30, tol=0.
 ↪001)
    lp_model.fit(X_train_complete, y_train_complete_missing)
    y_train_complete = lp_model.predict(X_train_complete)
    clf = SVC()
    clf.fit(X_train_complete, y_train_complete)
    y_pred = clf.predict(X_test)
    accuracy = accuracy_score(y_test, y_pred)

    return accuracy
```

[38]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_2_solution()


print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, float
), f"Task {task_id}: Your function should return a float "




del stu_ans
```

```
Task 2 - AG tests
Task 2 - your answer:
0.92
```

How confident would you be in eating a mushroom predicted with this rate of accuracy?